Category	Criteria	Excellent (10)	Good (8)	Satisfactory (6)	Needs Improvement (4 or below)
Database Design	Appropriateness of schema design (SQL/NoSQL), scalability, query efficiency, consistency, and alignment with use case	For SQL: Schema is normalized, relationships are clear, and queries are optimized for performance and scalability. For NoSQL: Schema uses denormalization, indexing, and efficient partitioning to achieve scalability and performance.	For SQL: Schema is mostly normalized but may have minor inefficiencies. For NoSQL: Schema is appropriate but may have minor inefficiencies in scalability or performance.	For SQL: Schema is functional but lacks normalization, with redundant data or inefficient queries. For NoSQL: Schema is functional but lacks scalability or has inefficient design.	Poor schema design: For SQL, redundant data or unclear relationships. For NoSQL, poorly structured schema or inefficient queries.
Scalability	System's ability to handle increased load, horizontal/vertical scaling, and resource optimization	Excellent use of horizontal scaling, load balancing, and resource optimization; system performs consistently under heavy load.	Good use of scaling techniques with minor inefficiencies; system handles moderate load effectively.	Basic scalability achieved but may have significant inefficiencies in performance under load.	Poor scalability design; system fails under load or lacks adequate resource optimization.
Project Complexity	Use of advanced features, problem-solving creativity, and depth of functionality	Implements advanced features with creative solutions and significant functionality; goes beyond course requirements.	Implements some advanced features; overall functionality is moderately complex and aligns with course requirements.	Project has basic functionality with minimal complexity or creativity.	Lacks complexity; overly simplistic or fails to meet basic project requirements.
Code Cleanliness	Readability, adherence to best practices, use of comments, and organization	Code is clean, modular, well-organized, properly commented, and adheres to all best practices.	Code is mostly clean and organized, with minor readability or commenting issues.	Code is functional but disorganized or inconsistently adheres to best practices.	Code is poorly organized, lacks comments, or is difficult to read and maintain.
Backend API Design	REST/gRPC standards, endpoints design, error handling, and security	APIs follow REST/gRPC standards with intuitive endpoints, comprehensive error handling, and strong security.	APIs mostly follow standards, with minor issues in endpoint design, error handling, or security.	APIs are functional but lack adherence to standards, error handling, or proper security.	APIs are poorly designed, insecure, or fail to meet REST/gRPC conventions.
Frontend Client Code	Functionality, UI/UX design, responsiveness, and integration with backend	Fully functional frontend with polished, responsive UI/UX; seamlessly integrates with backend APIs.	Functional frontend with good UI/UX, but may have minor design or responsiveness issues.	Frontend is minimally functional, with basic UI/UX and integration issues.	Frontend is incomplete, poorly designed, or fails to integrate properly with the backend.
Cloud Deployment	Containerization, scalability, cost optimization, use of cloud-native services, and CI/CD pipelines	Fully containerized deployment with an automated CI/CD pipeline; leverages cloud-native services for scalability and cost efficiency.	Deployed using containerization and cloud-native services; CI/CD pipeline is functional but may lack some optimizations.	Deployment is functional but lacks scalability, cost optimization, or proper CI/CD pipelines.	Deployment is incomplete, not containerized, or lacks functionality; poorly uses cloud resources.
Innovation	Creativity and originality of the project idea	Highly innovative project solving unique problems or showcasing original solutions.	Moderately creative project with some original ideas or approaches.	Functional but lacks creativity or significant innovation.	Unoriginal or derivative project idea with no innovation.
Documentation	README file, setup instructions, architecture overview, and inline documentation	Comprehensive documentation with clear setup instructions, architecture diagrams, and inline comments for code.	Good documentation with clear setup instructions and general architecture overview.	Basic documentation provided but lacks details (e.g., unclear setup or missing architecture overview).	Missing or insufficient documentation, making the project difficult to understand or set up.
Testing	Unit tests, integration tests, and test coverage	Comprehensive testing with high coverage for all critical functionalities; tests are automated and reliable.	Good testing with adequate coverage; minor gaps in test automation or coverage.	Limited testing with minimal coverage; some critical functionalities are not tested.	Testing is insufficient, unreliable, or absent.
Teamwork	Collaboration and division of tasks	Excellent collaboration with fair task division	Good teamwork with minor issues in task division	Basic collaboration but significant issues in task division	Poor collaboration, lack of teamwork, or unclear division of responsibilities.